

SIMATS ENGINEERING

ASSESSMENT TOOL 1

COURSE NAME: Artificial Intelligence

COURSE CODE: CSA1708

COURSE OUTCOME COVERED

C05: *Design AI-based systems using PySpark, RDD operations, and intelligent agent architectures, using scalable systems and integrate components in distributed AI applications. (BTL 5)*

Assessment Tool Weightage: 50%

QUESTIONS: 5

Annexure A

Duration : 1 Hour
Total Marks:50

Design Task

Code of Conduct:

I B VIJAYKARTHIKKAYAN(192511348) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

Design Task

1. Hybrid AI Recommendation Engine Design

Design a hybrid AI-based recommendation system that integrates content-based and collaborative filtering techniques using PySpark RDDs. Explain how distributed data processing can be used to handle large-scale user-item interactions, and describe the role of intelligent agents in adapting recommendations based on user behavior and feedback in real time.

2. Smart Disaster Detection System

Develop a distributed AI system using PySpark and RDD operations to detect natural disasters (such as floods or earthquakes) from multi-source data streams (satellite images, sensors, and social media). Describe the system architecture, data fusion techniques, and how intelligent agents coordinate to provide early warnings and response strategies.

3. AI-Based Fraud Detection Framework

Design a scalable fraud detection system using PySpark RDDs to process large volumes of financial transactions. Explain how anomaly detection algorithms can be implemented in a distributed manner and how intelligent agents collaborate to identify suspicious patterns, trigger alerts, and continuously improve detection accuracy.

4. Autonomous Supply Chain Optimization System

Construct a distributed AI system using PySpark for optimizing supply chain operations (inventory, logistics, demand forecasting). Describe how RDD transformations support large-scale data processing and how multiple intelligent agents interact to make decentralized decisions for cost minimization and efficiency improvement.

5. Personalized Learning Analytics Platform

Design an AI-driven educational platform that uses PySpark and RDDs to analyze student learning behavior and performance data. Explain how the system supports adaptive learning through intelligent agents, real-time feedback, and scalable analytics to personalize content delivery for thousands of learners.

RUBRICS FOR CALCULATION

Evaluation Criteria	Problem Understanding & Requirement Analysis	System Architecture Design	Use of PySpark & RDD Operations	Intelligent Agent Integration	Scalability & Distributed Computing Design	Data Flow & Processing Pipeline	Innovation & Creativity	Clarity, Presentation & Documentation
Weightage	10%	20%	15%	15%	10%	10%	10%	10%

Criteria	Excellent (5)	Good (4)	Average (3)	Poor (2)
Problem Understanding & Requirement Analysis	Clearly defines problem, objectives, and constraints	Mostly clear with minor gaps	Basic understanding	Unclear or incorrect
System Architecture Design	Complete, scalable, well-structured architecture with diagrams	Good design with minor issues	Basic design, lacks detail	Poor/incomplete
Use of PySpark & RDD Operations	Efficient and correct use of RDD transformations/actions	Mostly correct usage	Limited or partial use	Incorrect/missing
Intelligent Agent Integration	Strong integration with clear agent roles and logic	Good integration	Basic integration	Poor/no integration
Scalability & Distributed Computing Design	Clearly addresses scalability, fault tolerance, parallelism	Covers most aspects	Limited consideration	Not addressed
Data Flow & Processing Pipeline	Clear, logical, and well-defined data flow	Mostly clear	Some gaps	Poorly defined

Innovation & Creativity	Highly innovative, realistic, and impactful solution	Some innovation	Basic idea	No originality
Clarity, Presentation & Documentation	Very clear, neat diagrams, well-organized report	Mostly clear	Somewhat organized	Poor presentation

Question 1: Hybrid AI Recommendation Engine Design

1. Problem Understanding & Requirement Analysis

Modern recommendation platforms (e-commerce, streaming, ed-tech) must generate personalized suggestions for millions of users interacting with millions of items, in near real time, while handling cold-start users/items and sparse rating matrices. A single-technique recommender is insufficient:

- Collaborative filtering (CF) suffers from the cold-start problem for new users/items.
- Content-based (CB) filtering ignores collective community behaviour and over-specializes recommendations.
- The system must scale horizontally as the user-item interaction matrix grows into billions of entries.

Requirements: (a) distributed batch + streaming processing of interaction logs, (b) a hybrid scoring model that blends CB and CF signals, (c) an adaptive layer that updates recommendations from live feedback, and (d) fault-tolerant, horizontally scalable compute.

2. System Architecture Design

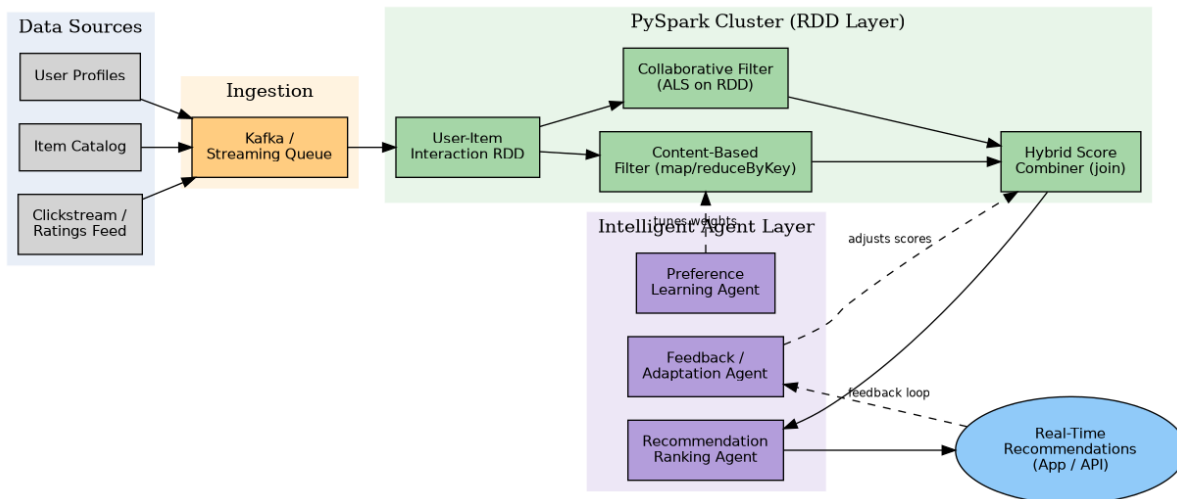


Figure 1: Hybrid recommendation engine architecture — ingestion, PySpark RDD processing, and the intelligent agent layer.

The architecture has four layers:

- Data Source Layer — user profiles, item catalog, and clickstream/rating events.
- Ingestion Layer — a Kafka-based streaming queue buffers high-velocity interaction events before they reach the cluster.
- PySpark RDD Processing Layer — builds the user-item interaction RDD and computes content-based similarity and collaborative-filtering scores in parallel.
- Intelligent Agent Layer — learns preferences, adapts scores from feedback, and ranks the final recommendation list delivered through an API.

3. Use of PySpark & RDD Operations

Interaction data is loaded as an RDD partitioned by user ID so that per-user computation is co-located, minimizing shuffles.

```
interactions = sc.textFile("hdfs://logs/interactions/*") \
    .map(lambda l: l.split(",")) \
    .map(lambda r: (r[0], r[1], float(r[2]))) # (user, item, rating)

# Content-based similarity: map items to feature vectors, then cosine similarity
item_features = sc.textFile("hdfs://catalog/items.csv").map(parse_item_features)
cb_scores = interactions.map(lambda x: (x[1], x)) \
```

```

        .join(item_features) \
        .map(compute_cb_score)

# Collaborative filtering: build (user, item, rating) RDD -> ALS.train()
from pyspark.mllib.recommendation import ALS
model = ALS.train(interactions.map(lambda r: (r[0], r[1], r[2])), rank=10, iterations=10)
cf_scores = model.predictAll(user_item_pairs)

# Hybrid combination via join + weighted reduceByKey
hybrid = cb_scores.join(cf_scores) \
    .mapValues(lambda s: 0.4*s[0] + 0.6*s[1]) \
    .reduceByKey(lambda a, b: max(a, b))

```

Key RDD transformations used: map (feature extraction), join (merging CB and CF scores), reduceByKey (aggregating per user-item pair), and mllib's ALS which internally relies on lazy RDD lineage and in-memory caching (persist()/cache()) for iterative optimization.

4. Intelligent Agent Integration

- Preference Learning Agent — continuously updates a user's latent preference vector from implicit signals (clicks, dwell time) and tunes the CB/CF weighting per user segment.
- Feedback / Adaptation Agent — observes accept/reject/skip actions on served recommendations and re-scores the candidate list in near real time (reinforcement-style reward signal).
- Recommendation Ranking Agent — applies business rules (diversity, novelty, fairness) on top of the hybrid score before final delivery.

These agents operate autonomously and asynchronously, communicating through the shared hybrid-score RDD/state store, forming a lightweight multi-agent system layered above the batch computation.

5. Scalability & Distributed Computing Design

- Horizontal scaling — additional Spark worker nodes are added as interaction volume grows; RDD partitioning by user ID keeps computation local and shuffle-light.
- Fault tolerance — RDD lineage (DAG) allows automatic recomputation of lost partitions without checkpointing every stage.
- Caching — the ALS model and item-feature RDD are cached in memory since they are reused across many scoring requests.
- Elastic streaming — Kafka partitions and Spark Structured Streaming micro-batches allow the ingestion layer to scale independently of the batch scoring layer.

6. Data Flow & Processing Pipeline

Raw events → Kafka topic → Spark Streaming micro-batch → interaction RDD → parallel CB/CF scoring → hybrid combiner → agent re-ranking → recommendation API → user feedback captured and fed back into the interaction RDD for the next cycle (closed-loop pipeline).

7. Innovation & Creativity

The design's novelty lies in treating the hybrid weight (currently 0.4/0.6 in the example) as a per-user, agent-tuned parameter rather than a fixed global constant, and in closing the feedback loop so that recommendations improve continuously without a full retraining cycle, using incremental agent-driven re-scoring between batch ALS retrains.

Question 2: Smart Disaster Detection System

1. Problem Understanding & Requirement Analysis

Disasters such as floods and earthquakes must be detected within seconds to minutes from heterogeneous, high-volume, high-velocity data: satellite imagery, ground sensor telemetry, and unstructured social-media text. The system must fuse dissimilar modalities, tolerate noisy/missing data, and issue warnings before human analysts could manually review the same volume of data.

- Functional requirement: near real-time ingestion and fusion of image, sensor, and text streams.
- Non-functional requirement: low-latency alerting, fault tolerance, and horizontal scalability during disaster spikes (when data volume surges).

2. System Architecture Design

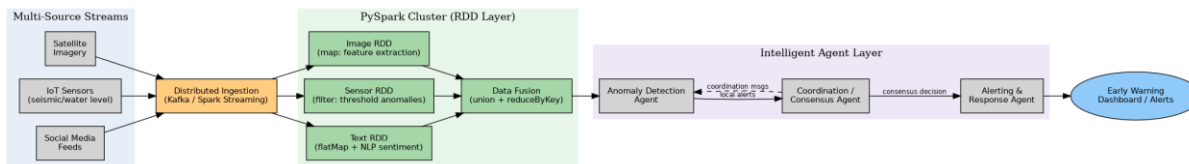


Figure 2: Multi-source disaster detection pipeline with PySpark fusion and coordinated agents.

- Multi-Source Layer — satellite imagery, IoT/seismic and water-level sensors, and social media feeds.
- Distributed Ingestion — Kafka topics per modality feed Spark Streaming.
- PySpark RDD Fusion Layer — each modality is transformed into a comparable feature RDD, then fused.
- Agent Layer — anomaly detection, cross-source coordination, and alerting/response agents.

3. Use of PySpark & RDD Operations

```
images = sc.binaryFiles("hdfs://satellite/*.png").map(extract_image_features) # map
sensors = sc.textFile("hdfs://sensors/*.csv") \
    .map(parse_reading) \
    .filter(lambda r: r.value > THRESHOLD) # filter
social = sc.textFile("hdfs://social/*.json") \
    .flatMap(tokenize) \
    .map(lambda w: (normalize(w), 1)) \
    .reduceByKey(lambda a, b: a + b) # flatMap + reduceByKey

# Data fusion: tag each RDD with (region, signal_type, score), then union + group
fused = images.union(sensors).union(social) \
    .map(lambda r: (r.region, r)) \
    .groupByKey() \
    .mapValues(fuse_region_signals)
```

`union()` merges heterogeneous streams once normalized to a common (region, signal_type, score) schema; `groupByKey/reduceByKey` aggregate multi-modal evidence per geographic region; `mapPartitions` can be used inside `fuse_region_signals` for batch image-processing efficiency.

4. Intelligent Agent Integration

- Anomaly Detection Agent — scores each region's fused signal against historical baselines to flag potential events.
- Coordination / Consensus Agent — cross-checks flags across modalities (e.g., sensor spike + matching social-media chatter) before escalating, reducing false positives.
- Alerting & Response Agent — dispatches warnings to authorities/citizens and recommends response actions (evacuation routes, resource allocation).

Agents communicate via message passing over the fused RDD's region key, forming a coordinated multi-agent early-warning system rather than a single centralized classifier.

5. Scalability & Distributed Computing Design

- Each modality is processed by an independently scalable Spark job so a spike in one source (e.g., social media during an event) does not bottleneck others.
- RDD lineage provides fault tolerance if a worker node fails mid-computation during a disaster surge.
- Checkpointing is applied to long streaming lineages to bound recovery time.

6. Data Flow & Processing Pipeline

Sensors/imagery/social data → Kafka → per-modality Spark Streaming jobs → normalized feature RDDs → union/fusion by region → anomaly scoring → agent consensus → alert dispatch → response-team dashboard, with a feedback channel that logs false/true alerts to retrain thresholds.

7. Innovation & Creativity

The design's innovation is cross-modal consensus before alerting — a flagged anomaly only becomes a warning once at least two independent modalities corroborate it, reducing false alarms while retaining the speed benefit of distributed streaming detection.

Question 3: AI-Based Fraud Detection Framework

1. Problem Understanding & Requirement Analysis

Financial institutions process millions of transactions per hour; fraud must be detected within a sub-second to few-second window to block a transaction before settlement, while keeping false-positive rates low to avoid blocking legitimate customers. The system must handle skewed, imbalanced data (fraud is rare) and evolving fraud patterns.

- Functional: real-time scoring of every transaction against learned fraud patterns.
- Non-functional: low latency, high throughput, continuous model improvement, auditability.

2. System Architecture Design

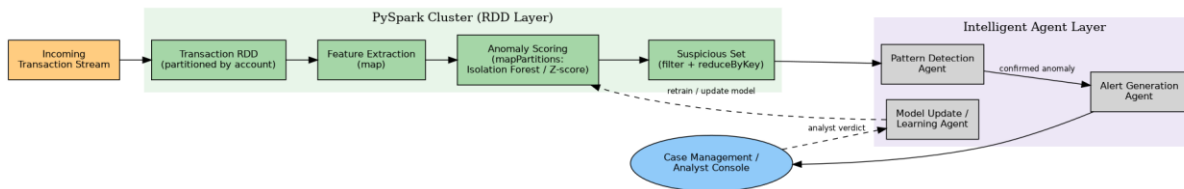


Figure 3: Fraud detection pipeline — RDD partitioning, anomaly scoring, and the agent feedback loop.

- Transaction Stream — incoming transactions partitioned by account ID for locality.
- PySpark RDD Layer — feature extraction, anomaly scoring, and filtering of suspicious cases.
- Agent Layer — pattern detection, alert generation, and a model-update/learning agent that closes the loop with analyst feedback.

3. Use of PySpark & RDD Operations

```
txns = sc.textFile("hdfs://transactions/*.csv") \
    .map(parse_txn) \
    .partitionBy(64, lambda t: hash(t.account_id)) # co-locate per-account history

features = txns.map(extract_features) # map: amount z-score, velocity, geo-distance

scores = features.mapPartitions(score_partition_isoforest) # mapPartitions: batch model scoring

suspicious = scores.filter(lambda s: s.score > FRAUD_THRESHOLD) \
    .map(lambda s: (s.account_id, s)) \
    .reduceByKey(keep_highest_score) # filter + reduceByKey
```

partitionBy keeps a customer's transaction history co-located to compute behavioural features without cross-node shuffles; mapPartitions applies the anomaly model in batches per partition (efficient for ML inference); reduceByKey deduplicates multiple flags for the same account into the highest-confidence case.

4. Intelligent Agent Integration

- Pattern Detection Agent — continuously mines the suspicious-transaction RDD for emerging fraud typologies (e.g., structuring, card testing).
- Alert Generation Agent — converts confirmed anomalies into prioritized case alerts for the analyst console.
- Model Update / Learning Agent — ingests analyst verdicts (true/false fraud) and triggers incremental retraining of the anomaly-scoring model, closing the loop back into the mapPartitions scoring stage.

5. Scalability & Distributed Computing Design

- Horizontal partitioning by account ID allows the cluster to scale linearly with transaction volume.
- In-memory caching of the trained model broadcast to all executors (Spark Broadcast variable) avoids re-shipping the model per partition.
- RDD lineage provides fault tolerance; checkpointing bounds recovery for long-running streaming jobs.

6. Data Flow & Processing Pipeline

Transaction stream → partitioned RDD → feature extraction → batch anomaly scoring → filtering of suspicious cases → pattern-detection agent → alert-generation agent → analyst console → verdicts fed to the learning agent → updated model broadcast back into the scoring stage (continuous-improvement loop).

7. Innovation & Creativity

The framework's innovation is the closed-loop learning agent that treats analyst feedback as a first-class data stream, allowing the anomaly model to adapt to new fraud typologies without a full offline retraining cycle, while broadcast variables keep inference latency low at scale.

Question 4: Autonomous Supply Chain Optimization System

1. Problem Understanding & Requirement Analysis

Supply chains involve inventory across many warehouses, logistics/routing decisions, and demand forecasting, all of which interact — an inventory shortfall affects routing, and forecast errors affect both. A centralized optimizer does not scale and cannot react quickly to local disruptions (e.g., a delayed shipment). The system needs decentralized, cooperating decision-makers over a large, continuously updated distributed dataset.

- Functional: forecast demand, optimize inventory levels, and plan logistics jointly.
- Non-functional: scalability across many warehouses/regions, resilience to local failures, cost minimization.

2. System Architecture Design

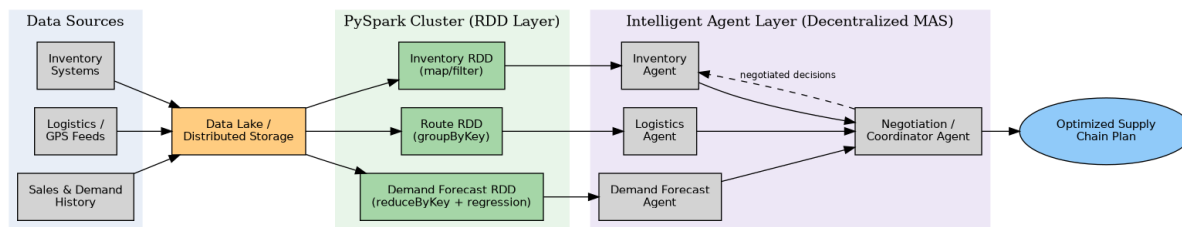


Figure 4: Distributed supply-chain optimization architecture with a decentralized multi-agent system (MAS).

- Data Sources — inventory systems, logistics/GPS feeds, and sales/demand history land in a distributed data lake.
- PySpark RDD Layer — separate RDDs for inventory, routing, and demand-forecast computation.
- Decentralized Agent Layer — an Inventory Agent, Logistics Agent, and Demand-Forecast Agent each own their domain, coordinated by a Negotiation/Coordinator Agent.

3. Use of PySpark & RDD Operations

```
inventory = sc.textFile("hdfs://inventory/*.csv") \
    .map(parse_stock) \
    .filter(lambda s: s.warehouse_active)           # map + filter

routes = sc.textFile("hdfs://logistics/gps/*.csv") \
    .map(parse_route_event) \
    .map(lambda r: (r.route_id, r)) \
    .groupByKey()                                   # groupByKey per route

demand = sc.textFile("hdfs://sales/history/*.csv") \
    .map(lambda r: (r.sku, r.units_sold)) \
    .reduceByKey(lambda a, b: a + b) \
    .mapValues(forecast_next_period)                # reduceByKey + forecasting
```

map/filter clean and select active warehouse records; groupByKey aggregates GPS events per delivery route for the logistics agent; reduceByKey followed by mapValues rolls up historical sales per SKU before forecasting — each RDD feeds one autonomous agent.

4. Intelligent Agent Integration

- Inventory Agent — monitors stock RDD and decides local replenishment thresholds per warehouse.
- Logistics Agent — plans routes from the grouped route RDD, minimizing transport cost/time.
- Demand Forecast Agent — supplies forward-looking demand estimates to both other agents.
- Negotiation/Coordinator Agent — resolves conflicts (e.g., inventory agent wants a large restock but logistics capacity is limited) via a bidding/negotiation protocol, producing a globally consistent plan from locally optimal proposals.

5. Scalability & Distributed Computing Design

- Each warehouse/region can run its own Inventory/Logistics agent pair, scaling out horizontally as new regions are added.
- Spark RDD partitioning aligns with warehouse/region boundaries so agents mostly operate on local partitions, reducing network overhead.
- Decentralization also improves resilience — a coordinator or single agent failure only degrades, not halts, the overall system, since other regional agents keep operating.

6. Data Flow & Processing Pipeline

Inventory/logistics/sales data → distributed data lake → parallel RDD pipelines (inventory, routing, forecasting) → per-domain agents produce local proposals → coordinator agent negotiates a global plan → optimized supply-chain plan executed → outcomes (delivery times, stockouts) logged back into the data lake for the next planning cycle.

7. Innovation & Creativity

The system's novelty is applying a decentralized, negotiation-based multi-agent protocol on top of Spark's data-parallel RDD pipelines — combining big-data-scale analytics with agent-based decentralized decision-making, rather than a single monolithic optimizer, which improves both scalability and fault tolerance.

Question 5: Personalized Learning Analytics Platform

1. Problem Understanding & Requirement Analysis

An adaptive learning platform must continuously analyze large volumes of clickstream, quiz, and engagement data from thousands of concurrent learners, identify each learner's mastery level and learning style, and personalize content delivery in real time. Requirements include scalable batch analytics for pattern discovery, low-latency personalization, and continuous adaptation from live feedback.

- Functional: profile each learner, recommend the next best content/activity, and adapt to performance changes.
- Non-functional: scale to thousands of concurrent learners, near real-time responsiveness, data privacy.

2. System Architecture Design

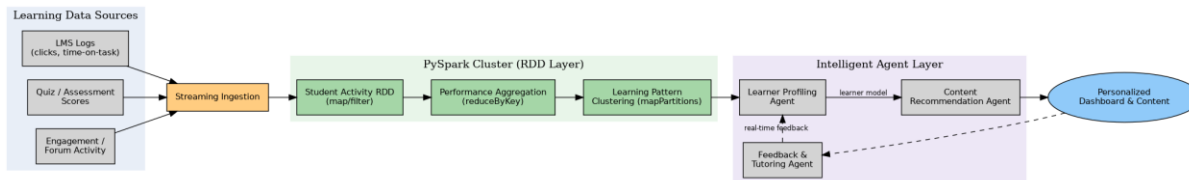


Figure 5: Personalized learning analytics architecture combining PySpark analytics with an adaptive agent layer.

- Learning Data Sources — LMS interaction logs, quiz/assessment scores, and forum/engagement activity.
- Streaming Ingestion — buffers high-frequency learner events.
- PySpark RDD Layer — cleans activity data, aggregates performance, and clusters learning patterns.
- Agent Layer — learner profiling, content recommendation, and feedback/tutoring agents personalize the experience.

3. Use of PySpark & RDD Operations

```
activity = sc.textFile("hdfs://lms/logs/*.json") \
    .map(parse_event) \
    .filter(lambda e: e.type in RELEVANT_EVENTS)      # map + filter

performance = activity.map(lambda e: (e.student_id, e.score)) \
    .reduceByKey(lambda a, b: a + b)                  # reduceByKey

patterns = activity.map(lambda e: (e.student_id, e.feature_vector)) \
    .groupByKey() \
    .mapPartitions(cluster_learning_styles)           # groupByKey + mapPartitions
```

filter removes noise events; reduceByKey aggregates per-student performance across sessions; groupByKey followed by mapPartitions clusters learners into behavioural/learning-style groups efficiently in batches — the resulting learner model feeds the agent layer.

4. Intelligent Agent Integration

- Learner Profiling Agent — maintains an up-to-date mastery/learning-style profile per student from the clustered pattern RDD.
- Content Recommendation Agent — selects the next-best content unit or activity matched to the learner's current profile.
- Feedback & Tutoring Agent — observes how the learner responds to recommended content and provides real-time hints/adjustments, feeding results back into the profiling agent.

5. Scalability & Distributed Computing Design

- Spark's partitioned RDDs allow analytics to scale as the learner base grows into the thousands/millions without redesigning the pipeline.

- Batch clustering (learning-style discovery) runs periodically at scale, while lightweight agent inference runs per-learner in near real time — separating heavy analytics from low-latency personalization.
- RDD lineage and caching (persisting the learner-profile RDD) provide fault tolerance and fast repeated access during a learning session.

6. Data Flow & Processing Pipeline

Learner events → streaming ingestion → activity RDD (map/filter) → performance aggregation (reduceByKey) → pattern clustering (groupByKey + mapPartitions) → learner-profiling agent → content-recommendation agent → personalized dashboard/content → learner interaction results captured as new events, closing the loop for continuous adaptation.

7. Innovation & Creativity

The platform's innovation is decoupling heavy, periodic batch analytics (learning-style clustering across the whole cohort) from lightweight, real-time agent-driven personalization per learner — giving both large-scale pattern discovery and instant adaptivity without re-running full-cohort analytics on every interaction.